Course: CSE302 Computer Organization and Architecture
Group Project: Five-cycle non-pipelined RISC processor using Verilog HDL

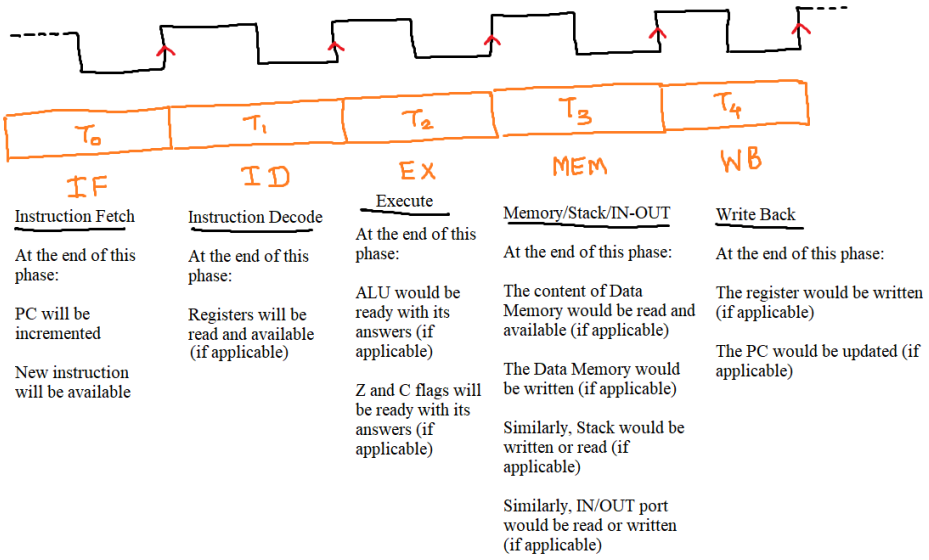
- Five-cycle non-pipelined RISC processor using Verilog HDL, for instruction set and instruction format given in this document
  - Logisim processor taken as reference
- All Flip Flops (FF) are clk based memory and have Reset

Instruction Set

Instruction Format (total 32-bits)

OPCODE [31:24]	**Destination [23:20]**	**Source-1 [19:16]**	**Source-2 [15:12]**	**IMM [11:0]** Used as: • ADDR = 12-bit direct address for DataMemory OR 12-bit direct address for Jump instructions • IMM [3:0] used as Source-3 (for reading R_{S3}) • IMM[7:0] used as Immediate-Data (referred to as IMMDATA) • IMM [3:0] used as the address for Input and Output Ports (referred to as ADDR4)

Timing Phases: Typical (IF/ID/EX/MEM/WB)



Acronyms:

- IMMDATA = IMM[7:0]
- ADDR = IMM[11:0]
- XP = X-Pointer (12-bits) = {R_{S1}[3:0], R_{S2}[7:0]}
- For finding R_{S3}, the Source-3 = IMM[3:0]
- ADDR4 = IMM[3:0]

OPCODE	**Mnemonic**	**Flags affected**
00H	NOP	None
01H	ADD $R_D \leftarrow R_{S1} + R_{S2}$	C, Z
02H	ADDI $R_D \leftarrow R_{S1}$ + IMMDATA	C, Z
03H	ADDC $R_D \leftarrow R_{S1} + R_{S2} + C$	C, Z
04H	SUB $R_D \leftarrow R_{S1}$ + 1'sComp(R_{S2}) + 1	C, Z
05H	SUBI $R_D \leftarrow R_{S1}$ + 1'sComp(IMMDATA) + 1	C, Z

06H	SUBC $R_D \leftarrow R_{S1} + 1'sComp(R_{S2}) + C$	C, Z
07H	CMP $R_{S1} + 1'sComp(R_{S2}) + 1$	C, Z
08H	CMPI $R_{S1} + 1'sComp(IMMDATA) + 1$	C, Z
09H	AND $R_D \leftarrow R_{S1}$ & R_{S2}	Z
0AH	ANDI $R_D \leftarrow R_{S1}$ & IMMDATA	Z
0BH	OR $R_D \leftarrow R_{S1} \mid R_{S2}$	Z
0CH	ORI $R_D \leftarrow R_{S1} \mid$ IMMDATA	Z
0DH	EXOR $R_D \leftarrow R_{S1}$ ^ R_{S2}	Z
0EH	EXORI $R_D \leftarrow R_{S1}$ ^ IMMDATA	Z
0FH	INC $R_D \leftarrow R_{S1}$ + 01H	Z
10H	DEC $R_D \leftarrow R_{S1} + 1'sComp(01H) + 1$	Z
11H	MOVI $R_D \leftarrow$ IMMDATA	
12H	MOV $R_D \leftarrow R_{S1}$	
13H	MOVF $R_D \leftarrow \{C, 6'b00000, Z\}$	
14H	LOAD $R_D \leftarrow M[ADDR]$	
15H	LOADI $R_D \leftarrow M[XP]$	
16H	STORE $M[ADDR] \leftarrow R_{S1}$	
17H	STOREI $M[XP] \leftarrow R_{S3}$	
18H	LRS $R_D[7:0] \leftarrow \{1'b0, R_{S1}[7:1]\}$ (logical shift right by 1-bit)	$C = R_{S1}[0]$, Z
19H	ARS $R_D[7:0] \leftarrow \{R_{S1}[7], R_{S1}[7:1]\}$ (arithmetic shift right by 1-bit)	$C = R_{S1}[0]$, Z
1AH	LLS $R_D[7:0] \leftarrow \{R_{S1}[6:0],1'b0\}$ (logical shift left by 1-bit)	$C = R_{S1}[7]$, Z
1BH	ROTR $R_D[7:0] \leftarrow \{R_{S1}[0], R_{S1}[7:1]\}$ (rotate right by 1-bit)	$C = R_{S1}[0]$, Z
1CH	ROTL $R_D[7:0] \leftarrow \{R_{S1}[6:0], R_{S1}[7]\}$ (rotate left by 1-bit)	$C = R_{S1}[7]$, Z
1DH	ROTRC $\{C, R_D[7:0]\} \leftarrow \{R_{S1}[0], C, R_{S1}[7:1]\}$ (rotate right with C by 1-bit)	$C = R_{S1}[0]$, Z
1EH	ROTLC $\{C, R_D[7:0]\} \leftarrow \{R_{S1}[7:0], C\}$ (rotate left with C by 1-bit)	$C = R_{S1}[7]$, Z
1FH	JMP**Z** ADDR	
20H	JMP**NZ** ADDR	
21H	JMP**C** ADDR	
22H	JMP**NC** ADDR	
23H	CALL ADDR	
24H	RET	
25H	PUSH Stack $\leftarrow R_{S1}$	
26H	POP $R_D \leftarrow$ Stack	
27H	unsignedMULT $\{Stack, R_D\} \leftarrow R_{S1} \times R_{S2}$ (lower-order byte written in WB to RD) (higher-order byte written to Stack in MEM) The programmer may in subsequent instruction POP to a register the high-order byte of the answer	
28H	IN $R_D \leftarrow PORT[ADDR4]$ Input ports have address 0H, 1H, 2H, 3H	
29H	OUT $PORT[ADDR4] \leftarrow R_{S1}$ Output ports have address 8H, 9H, AH, BH	
2AH	JMP ADDR (unconditional jump)	

Notes:

- There are 16 registers, R_0 to R_{15}, each register is 8-bits
- ALU is 8-bits; There are two operands, and each operand is 8-bits
- Address bus for Instruction memory is 12-bits
- Address bus for Data memory is 12-bits
- The internal address bus (and mechanism of SP) of Stack is 8-bits
- The Address bus for I/O ports are 4-bits

- Control signals are generated using a dedicated Control Logic module that decodes the opcode along with timing signals (T0–T4).

- The datapath uses multiple small multiplexers (2-to-1, 3-to-1, and 4-to-1) for operand selection, memory addressing, stack operations, and write-back selection. The selection inputs of these multiplexers are derived from logical conditions on the opcode rather than using the opcode directly as a multiplexer select input.

1. **Processor.v file with the following module**

Module Name	Inputs	Outputs	Comments
RISCprocessor	clk Reset [7:0] InpExtWorld1 [7:0] InpExtWorld2 [7:0] InpExtWorld3 [7:0] InpExtWorld4	[7:0] OutExtWorld1 [7:0] OutExtWorld2 [7:0] OutExtWorld3 [7:0] OutExtWorld4	(Overall Structural coding) Important instances are mentioned below: RegisterFile InstMEM4096 SRAM4096 Stack256 ALU8 TimingGen ProgCounter INport OUTport ControlLogic & some MUX32to1_8bit

2. **Subcomponents.v file with the following modules**

Module Name	Inputs	Outputs	Comments
eightbitRegwithLoad	Clk Reset load [7:0] Datain	[7:0] Dataout	This module is also used as an 8-bit FF by simply passing 1'b1 to load when instantiating it in another module
RegisterFile	Clk Reset RegFileRead RegFileWrite [7:0] Datain [3:0] Source1 [3:0] Source2 [3:0] Source3 [3:0] Destin	[7:0] Dataout1 [7:0] Dataout2 [7:0] Dataout3	Structural coding, with some important instances: 16 instances of eightbitRegwithLoad 1 instance of Decoder4to16_withE 3 instances of MUX16to1_8bit 3 instances of eightbitRegwithLoad Dataout3 used for STOREI instruction
InstMEM4096	Clk Reset [11:0] Address InstRead	[31:0] Dataout [7:0] Opcode [3:0] Destin [3:0] Source1 [3:0] Source2 [3:0] Source3 [11:0] ADDR [7:0] ImmData [3:0] ADDR4	(Behavioral coding) The memory variable is declared as: reg [31:0] instmemory [0:4095] Initial block and $readmemb used to load assembly language program (instructions in binary format)
SRAM256	Clk Reset [7:0] Address	[7:0] Dataout	(Behavioral coding) The memory variable is declared as:

	SRAMRead SRAMWrite [7:0] Datain		reg [7:0] datamemsmall [0:255]
SRAM4096	Clk Reset [11:0] Address SRAMRead SRAMWrite [7:0] Datain	[7:0] Dataout	(Behavioral coding) The memory variable is declared as: reg [7:0] datamemlarge [0:4095]
Stack256	Clk Reset StackRead StackWrite [7:0] Datain	[7:0] Dataout	Partial structural coding, with the following important instances: 1 instance of SRAM256 2 instances of MUX4to1_8bit
ALU8	Clk Reset [7:0] Operand1 [7:0] Operand2 [7:0] Opcode ALUSave ZflagSave CflagSave	Zflag Cflag [7:0] ALUout [7:0] ALUout2	ALUout2 used for storing the high-order byte during UMULT instruction
TimingGen	Clk Reset	T0 T1 T2 T3 T4	
ProgCounter	Clk Reset PCenable PCupdate [11:0] Address	[11:0] PC	
INport	Clk Reset INportRead [7:0] InpExtWorld1 [7:0] InpExtWorld2 [7:0] InpExtWorld3 [7:0] InpExtWorld4 [3:0] Address	[7:0] Dataout	
OUTport	Clk Reset [3:0] Address [7:0] Datain OUTportWrite	[7:0] OutExtWorld1 [7:0] OutExtWorld2 [7:0] OutExtWorld3 [7:0] OutExtWorld4	
ControlLogic	Clk Reset T1 T2 T3 T4 Zflag Cflag [7:0] Opcode	PCupdate SRAMRead SRAMWrite StackRead StackWrite ALUSave ZflagSave CflagSave INportRead OUTportWrite RegFileRead RegFileWrite	